

Data Poisoning Attack Assignment

This assignment will walk you through how to carry out a targeted data poisoning attack on an ML computer vision model, ResNet50.

Recommended: Run the cells as you complete them to troubleshoot easier. Only focus on TODO comments

Data poisoning involves injecting manipulated data into the training dataset to degrade the model's performance on specific classes. In a targeted poisoning attack, the adversary modifies a subset of the training data from a chosen victim class, assigning them misleading yet high-confidence labels. This manipulation leads to the model misclassifying instances of the poisoned class. We will be performing a targeted data poisoning attack.

Setup

For the sake of this exercise, we have provided the dataset and model setup for you, but if you would like more information, you can reference the notebook.

Import Dependencies

We have imported all the dependencies you will need for this assignment such as the [adversarial robustness toolbox \(ART\)](#) and torch libraries and modules. You can take a look at the documentation for each library that we use if you wish.

Format Data

Please copy the TrafficDetectionDataset.zip to your My Drive directory.

Now, we need to import the dataset and format the data. Within the assignment folder, we have provided you a dataset that has already been normalized to work properly with the ResNet50 model. You can take a closer look at the functions if you would like to learn more about data preprocessing.

Load the Model

As mentioned previously, we will be using a pretrained ResNet50 model to work with our dataset.

Please ensure that the correct file path to your assignment folder is written.

Near the end of the Setup section, you will need to complete the Classifier function call for the classifier we will use on our model.

PyTorchClassifier() Function:

You will need to complete the classifier function to advance. The missing parameters are the loss function and the optimizer we will use.

```
classifier = PyTorchClassifier(clip_values=(0.0, 1.0),
                              model=model,
                              nb_classes=58,
                              input_shape=(3,128,128),
                              loss=#Loss Function,
                              optimizer=#Optimization Algorithm)
```

Pass in the values we have set above as the function parameters.

You don't have to complete code here but it is good to know what's going on when we train ("fit") the model on our data.

```
classifier.fit(
    trainImages,
    trainLabels,
    nb_epochs=10,
    training_mode=True,
    batch_size=25,
    verbose=True
)
```

- We first have to pass in our data as images and labels for the model to learn.
- Epoch: A complete passthrough of the dataset through the artificial neural network.
 - This is very important in deep learning methods as it allows the model to get better at making predictions by adjusting its weights.
 - Adjust this number delicately to avoid underfitting or overfitting the model.

Attack

Setup

To perform a data poisoning attack we need to select a target class and desired class.

- Target Class: This is the class we want to poison and cause the model to misidentify.
- Desired Class: We will create the poisoned images and labels with this class to cause a misidentification of the target class.

Configure the Attack

You don't have to complete code here.

We will be using ART's feature collision attack function to simulate the data poisoning attack.

I've described a few notable parameters but the documentation for this function is linked below.

```
attack = FeatureCollisionAttack(classifier,
                                targetInstance,
                                featureLayer,
                                max_iter=100,
                                similarity_coeff=256,
                                watermark=0.3,
                                learning_rate=1)

# Create poison instances
poison, poisonLabels = attack.poison(baseInstances)
```

FeatureCollisionAttack function:

- classifier: specifies the classifier that is going to be attacked.
- targetInstance: The target input to misclassify at test time; poisoned.
- watermark: sets the potency of the attack on the clean samples.

You can make the watermark value higher and see how that changes the output images after poisoning.

Docs:

<https://adversarial-robustness-toolbox.readthedocs.io/en/latest/modules/attacks/poisoning.html>

Now, in the notebook we will display the poison images we will use on the target class.

Train with Poison

Remember how we trained our model with the clean images and labels above? Now, we will use the poisoned images and labels we created from our Feature Collision Attack to pass in as the parameters for this function.

```
classifier.fit(  
    #Images,  
    #Labels,  
    nb_epochs=20,  
    batch_size=2,  
    verbose=True  
)
```

Pass in the values of the poison instances we made earlier as the function parameters.

Test the Attack

Check your results against this output. Your model should now be predicting incorrect labels against the poisoned class.

The model is now incorrectly identifying the target class as the desired class which we used to trick the model with.



Questions

1. We've learned that the watermark affects the transparency of the attack signature on the clean sample. What is one downside of increasing the watermark value on the attack function?
2. In this assignment we only trained with the poison images, what can you expect to happen if we added the poison in but trained on the entire dataset? Will the attack be more or less effective? Will it be harder to catch?

"Garbage in, garbage out" (Babbage, 1864; Mellin, 1957)

3. How is this famous saying in the world of AI relevant here in the context of data poisoning attacks?

Read More

Read more on targeted vs untargeted poisoning attacks:

- <https://www.restack.io/p/federated-learning-answer-local-model-poisoning-attacks-cat-ai>

If you want to read more on Data Poisoning attacks, please refer to these documents linked below.

- CrowdStrike Data Poisoning article:
<https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/data-poisoning/>
- NIST Data Poisoning article:
https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=934932

If you wish to learn more, these are very frequently used in Deep Learning methods:

- Cross Entropy Loss Function:
<https://www.geeksforgeeks.org/what-is-cross-entropy-loss-function/>
- Adam Optimizer: <https://pytorch.org/docs/stable/generated/torch.optim.Adam.html>

Poison attack docs:

- <https://adversarial-robustness-toolbox.readthedocs.io/en/latest/modules/attacks/poisoning.html>